ESTRUTURA DE DADOS · JAVASCRIPT

# Tabelas Hash

Como achar um dado sem procurar por ele




Professor Mestre Alexandre Gomes

ROTEIRO DA AULA

# A história de uma ideia

1

**Problema**
como achar um dado entre milhões?

2

**Desejo**
ir direto à posição, sem procurar.

3

**Ideia**
calcular o índice a partir da chave.

4

**Obstáculo**
duas chaves podem cair na mesma posição.

5

**Solução**
tratar colisões e manter a tabela folgada.

6

**Prática**
construir, medir e resolver problemas.

O PROBLEMA

# Onde está o que eu procuro?

**1.000.000**

de alunos cadastrados.

Como achar a **Maria Silva**?

O PROBLEMA

# Três jeitos de procurar

| Estratégia | Como funciona | Comparações em 1 milhão |
|---|---|---|
| Busca linear | Olhar um por um | até 1.000.000 |
| Busca binária | Exige ordenar e corta o espaço ao meio | cerca de 20 |
| Acesso direto | Ir direto à posição | 1 |

## E se eu soubesse a posição da Maria?

A IDEIA

# O guarda-volumes

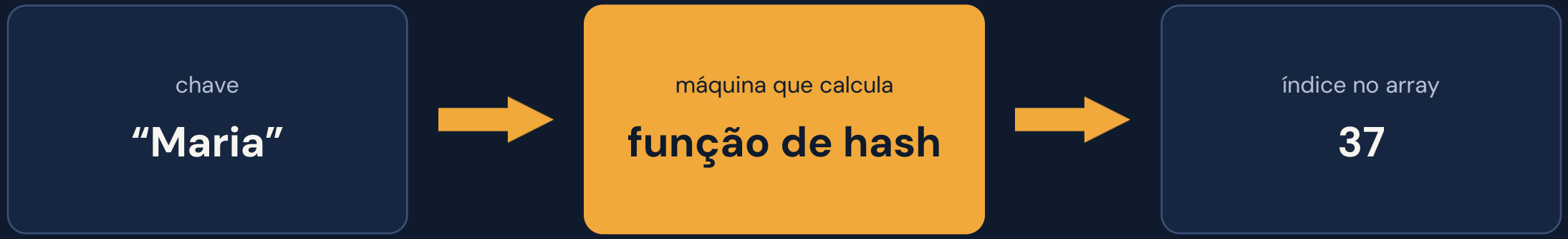



Recebi o armário 37. Para pegar a mochila, vou direto no 37.

Em um array, se sei o índice, **alunos[37]** é instantâneo: O(1).

A IDEIA

# Da chave ao índice




**Em vez de procurar o dado, eu calculo onde ele está.**

A IDEIA

# Vocabulário da aula

### Chave

O que identifica o dado: nome, CPF, matrícula.

### Função de hash

A máquina que converte a chave em um número.

### Índice

A posição do array onde o dado fica.

### Tabela hash

O array e a função de hash trabalhando juntos.

FUNÇÃO DE HASH · NÚMEROS

# O resto da divisão

`2024015 % 10`

**5**

`2024387 % 10`

**7**

`2024922 % 10`

**2**

O **%** garante que o resultado sempre cabe no array: com tamanho T, o resto fica entre 0 e T-1.

**Analogia do relógio: depois do 12, volta ao 1.**

FUNÇÃO DE HASH · TEXTO

# Toda letra é um número

| a | b | c | M |
|---|---|---|---|
| 97 | 98 | 99 | 77 |

```
console.log("a".charCodeAt(0)); // 97
console.log("Maria".charCodeAt(0)); // 77
```

PRIMEIRA FUNÇÃO DE HASH

# Somar os códigos das letras

```
// Versão ingênua: soma dos códigos
function hashSimples(chave, tamanho) {
  let soma = 0;
  for (let i = 0; i < chave.length; i++) {
    soma += chave.charCodeAt(i);
  }
  return soma % tamanho;
}

console.log(hashSimples("abc", 11)); // 8
```

Conta: 97 + 98 + 99 = 294, e 294 % 11 = 8. A palavra “abc” vai para a posição 8.

O TESTE QUE QUEBRA A IDEIA

# A soma ignora a ordem

**Anagramas**

“abc” → 8

“cba” → 8

**Coincidência da soma: 65 + 68 = 66 + 67**

“AD” → 1

“BC” → 1

**O que aconteceu? Mesma posição para chaves diferentes.**

UMA FUNÇÃO MELHOR

# Dar peso à posição

**Analogia dos dígitos**

## 123 ≠ 321

Os mesmos dígitos, valores diferentes: cada posição tem um peso (centena, dezena, unidade).

**A mesma ideia com letras**

```
hash = 0
para cada letra:
  hash = hash * 31 + código
```

Multiplicar o acumulado antes de somar a próxima letra faz a ordem importar.

UMA FUNÇÃO MELHOR

# A conta passo a passo, tamanho 11

### “abc”

```
após 'a': 0 * 31 + 97 = 97
após 'b': 97 * 31 + 98 = 3105
após 'c': 3105 * 31 + 99 = 96354
96354 % 11 = 5
```

### “cba”

```
após 'c': 0 * 31 + 99 = 99
após 'b': 99 * 31 + 98 = 3167
após 'a': 3167 * 31 + 97 = 98274
98274 % 11 = 0
```

**“abc” vai para a posição 5 e “cba” para a posição 0. Agora a ordem importa.**

UMA FUNÇÃO MELHOR

# A função polinomial

```
// Versão melhor: polinomial
function hashPolinomial(chave, tamanho) {
  let h = 0;
  for (let i = 0; i < chave.length; i++) {
    h = (h * 31 + chave.charCodeAt(i)) % tamanho;
  }
  return h;
}

console.log(hashPolinomial("abc", 11)); // 5
console.log(hashPolinomial("cba", 11)); // 0
```

**Também separa**

**“AD” → 4**

**“BC” → 1**

O % dentro do laço evita números gigantes em chaves longas.

UMA FUNÇÃO MELHOR

# O que faz uma boa função

### Determinística

A mesma chave sempre dá o mesmo índice, como o armário de cada pessoa.

### Rápida

Calcular tem que ser mais barato que procurar.

### Uniforme

Espalha as chaves por todo o array, como convidados por todas as mesas.

### Usa toda a chave

Cada parte influencia. Não vale olhar só a primeira letra do nome.

O OBSTÁCULO

# Colisão

Duas chaves diferentes resultam no **mesmo índice**.




**O armário 5 já está ocupado. E agora?**

O OBSTÁCULO

# Colisões são inevitáveis

Exemplo pequeno

## 12 chaves, 11 posições

Se você tem 12 objetos e só 11 gavetas, alguma gaveta vai ficar com 2 objetos. Não tem como fugir.

Na vida real

## Infinitas chaves

Podem existir infinitos nomes possíveis, mas o array tem um número fixo de posições. Mesmo aumentando o array, sempre há mais chaves possíveis do que posições.

**Colisão não é bug. O trabalho é lidar bem com ela.**

TRATANDO COLISÕES

# Encadeamento




## Cada posição guarda uma lista.

Se duas pessoas caem no armário 5, as duas mochilas ficam lá dentro. Na busca, confiro só a lista do armário 5.

**Função boa, listas curtas.**

ENCADEAMENTO · CÓDIGO 1 DE 4

# A classe HashTable

```
class HashTable {
  constructor(tamanho = 11) {
    this.tamanho = tamanho;
    this.tabela = Array.from({ length: tamanho }, () => []);
    this.quantidade = 0;
  }
  _hash(chave) {
    let h = 0;
    for (let i = 0; i < chave.length; i++) {
      h = (h * 31 + chave.charCodeAt(i)) % this.tamanho;
    }
    return h;
  }
```

**O que guarda**

**tabela**: array de listas, uma por posição.

**quantidade**: itens guardados, para o fator de carga.

Nos exemplos: **chave** = nome do aluno (única, é o que se procura); **valor** = a nota dele.

ENCADEAMENTO · CÓDIGO 2 DE 4

# set: inserir ou atualizar

```
set(chave, valor) {
  const balde = this.tabela[this._hash(chave)];
  for (const par of balde) {
    if (par[0] === chave) {
      par[1] = valor; // já existe: atualiza
      return;
    }
  }
  balde.push([chave, valor]);
  this.quantidade++;
  if (this.quantidade / this.tamanho > 0.75) this._rehash();
}
```

**Passo a passo**

1. Calcula a posição e pega a lista.
2. Se a chave já existe, só troca o valor (a nota).
3. Senão, acrescenta ao fim.

ENCADEAMENTO · CÓDIGO 3 DE 4

# get: buscar pela chave

```
get(chave) {
  const balde = this.tabela[this._hash(chave)];
  for (const [k, v] of balde) {
    if (k === chave) return v;
  }
  return undefined;
}
```

```
const t = new HashTable(11);
t.set("Ana", 8);
t.set("Eva", 6);
t.get("Eva"); // 6
t.set("Ana", 9); // corrige
t.get("Ana"); // 9
t.get("Zeca"); // undefined
```

A busca percorre só a lista da posição calculada, não a tabela inteira.

ENCADEAMENTO · CÓDIGO 4 DE 4

# delete: remover uma chave

```
delete(chave) {
  const balde = this.tabela[this._hash(chave)];
  const i = balde.findIndex(([k]) => k === chave);
  if (i === -1) return false;
  balde.splice(i, 1);
  this.quantidade--;
  return true;
}
```

```
t.delete("Eva"); // true
t.get("Eva"); // undefined
t.delete("Zeca"); // false
```

Remover só mexe na lista da própria posição: o resto da tabela não é tocado.

MANTER A TABELA FOLGADA

# Fator de carga

**fator de carga = itens ÷ tamanho do array**

5 itens em 10 posições



**0,5: meio cheio**

9 itens em 10 posições



**0,9: quase lotado**

Como uma sala de aula: vazia, é fácil achar lugar; lotada, todo mundo se esbarra. Mais carga, mais colisões, mais lentidão.

MANTER A TABELA FOLGADA

# Rehash

```
_rehash() {
  const antiga = this.tabela;
  this.tamanho = this.tamanho * 2 + 1;
  this.tabela = Array.from(
    { length: this.tamanho }, () => []);
  this.quantidade = 0;
  for (const balde of antiga) {
    for (const [k, v] of balde) this.set(k, v);
  }
}
```

**Carga acima de 0,75 no encadeamento**

1. Cria um array maior.
2. Recalcula o hash de todas as chaves.
3. Reposiciona cada item.

**Mudou o tamanho, mudou o índice.**

MANTER A TABELA FOLGADA

# Quanto custa, no fim

| Operação | Médio | Pior caso |
|---|---|---|
| Inserir | O(1) | O(n) |
| Buscar | O(1) | O(n) |
| Remover | O(1) | O(n) |

**Médio O(1):** função boa e tabela não lotada.

**Pior O(n):** tudo colide na mesma posição e a tabela vira uma lista.

**Rehash:** custa O(n), mas é raro, então a média por inserção segue O(1).

**O hash troca memória por velocidade.**

HASH NATIVO DO JAVASCRIPT

# Object, Map e Set

```
// Object como dicionário
const alunos = { "123": "Ana", "456": "Bruno" };

// Map: chaves de qualquer tipo
const mapa = new Map();
mapa.set("123", "Ana");
mapa.set(42, "número");
console.log(mapa.get("123"), mapa.size, mapa.has(42));

// Set: conjunto sem duplicados
const unicos = new Set([1, 2, 2, 3, 3]);
console.log([...unicos]);
```

**Por baixo do capô**

Agora vocês sabem o que acontece em **mapa.get(chave)**.

Acesso médio O(1) nos três.

PRÁTICA · EM DUPLAS: 1 A 5 AGORA, 6 DE TAREFA

# Seis problemas clássicos

| # | Problema | Estrutura | Sem hash → com hash |
|---|---|---|---|
| 1 | Detectar duplicados | Set | $O(n^2) \rightarrow O(n)$ |
| 2 | Frequência de palavras | Map | $O(n \cdot k) \rightarrow O(n)$ |
| 3 | Two Sum | Map | $O(n^2) \rightarrow O(n)$ |
| 4 | Primeiro caractere único | Map | $O(n^2) \rightarrow O(n)$ |
| 5 | Agrupar anagramas | Map | $O(n^2) \rightarrow O(n \cdot k \log k)$ |
| 6 | Fibonacci com memoização | Map | $O(2^n) \rightarrow O(n)$ |

ENCERRAMENTO

# A história em cinco frases

1. Procurar é caro, e o desejo é ir direto à posição.
2. A função de hash transforma a chave em índice: calculamos onde o dado está.
3. Colisões são inevitáveis (casa dos pombos) e precisam de tratamento.
4. Encadeamento guarda uma lista por posição: buscar e remover só mexem na lista daquela posição.
5. Carga baixa e rehash mantêm o custo médio em O(1): trocamos memória por velocidade.